

Relatório Técnico — Trabalho Prático T5

Otimização Física de Consultas com Índices em PostgreSQL

Disciplina: SCC-541 – Laboratório de Bases de Dados (1º Semestre de 2026)

Docente: Prof. Dr. Caetano Traina Jr.

PAE: Gabriel Lucca de Melo

Grupo de Trabalho

Nome	NUSP	E-mail
Katiely Feitosa de Lacerda	12777100	katiely.lacerda@usp.br
Leonardo Gonzalez	15657074	leonardo.gonzalez@usp.br
Miguel Filippo Calhabeu	15480331	miguel.calhabeu@usp.br
Renan Silva Soriano	11794824	renan.soriano@usp.br
Guilherme Motta Tranche	13671549	guilherme.tranche@usp.br

1. Introdução e Metodologia Experimental

Este relatório apresenta os resultados práticos da otimização física de consultas na base de dados integrada de **Fórmula 1 + GeoNames (Cidades e Países)**, avaliando o ganho de desempenho obtido com a criação de índices estruturados.

Para medir os tempos de execução de forma precisa e mitigar ruídos do sistema operacional, os testes seguiram o seguinte protocolo:

1. **Aquecimento de Cache (Warm Cache):** Execução prévia das consultas para carregar os blocos de dados no Buffer Pool da memória RAM.
 2. **Medição Estatística (100 Execuções):** Criação das funções customizadas em PL/pgSQL (Mede_Tempo_Q1 e Mede_Tempo_Q2), que executam a respectiva consulta 100 vezes de forma sequencial e retornam a média aritmética de tempo em milissegundos (ms).
 3. **Análise do Plano Físico:** Inspeção do plano gerado pelo SGBD usando a instrução EXPLAIN ANALYZE.
-

2. Exercício 1 — Busca de Piloto por Nome Exato

A consulta busca recuperar a nacionalidade de um piloto a partir da concatenação de seu nome (given_name) e sobrenome (family_name) correspondendo a uma igualdade exata:

```
SELECT (given_name || ' ' || family_name)::TEXT AS driver_name,  
nationality::TEXT  
FROM drivers  
WHERE (given_name || ' ' || family_name) = 'Ayrton Senna';
```

2.1. Estado Inicial (Sem Índice)

- **Tempo Médio por Execução: 0,07 ms**
- **Plano de Execução (EXPLAIN ANALYZE):**

```
Seq Scan on drivers (cost=0.00..10.53 rows=1 width=64) (actual
time=0.050..0.072 rows=1 loops=1)
  Filter: (((given_name)::text || ' '::text) || (family_name)::text) =
'Ayrton Senna'::text)
  Rows Removed by Filter: 615
Planning Time: 0.034 ms
Execution Time: 0.078 ms
```

- **Análise:** Como o filtro depende de uma expressão calculada e não havia índices apropriados, o PostgreSQL realizou uma varredura sequencial completa (Seq Scan) na tabela drivers. Ele leu em CPU todos os 616 registros para calcular a concatenação e filtrar a linha desejada, o que gera custo linear $O(N)$.

2.2. Escolha e Justificativa do Índice

Optamos pela criação de um **Índice Hash Funcional** baseado na expressão de busca:

```
CREATE INDEX idx_drivers_fullname_hash ON drivers USING hash (((given_name
|| ' ' || family_name)));
```

- **Justificativa:** O predicado da consulta utiliza estritamente o operador de igualdade exata (=). A estrutura Hash é teoricamente ótima para buscas pontuais de igualdade, resolvendo o mapeamento em tempo constante médio $O(1)$. A criação foi baseada em expressão (índice funcional) para evitar o recálculo da concatenação de strings em tempo de execução.
- **Nota de Design:** Chaves Hash no PostgreSQL não suportam índices de cobertura (INCLUDE) ou índices parciais com filtro (WHERE), impossibilitando armazenar a coluna nationality no próprio índice.

2.3. Estado Otimizado (Com Índice)

- **Tempo Médio por Execução: 0,01 ms** (Aceleração de aproximadamente **7 vezes**)
- **Plano de Execução (EXPLAIN ANALYZE):**

```
Bitmap Heap Scan on drivers (cost=4.02..9.73 rows=3 width=64) (actual
time=0.005..0.005 rows=1 loops=1)
  Recheck Cond: (((given_name)::text || ' '::text) ||
(family_name)::text) = 'Ayrton Senna'::text)
  Heap Blocks: exact=1
   -> Bitmap Index Scan on idx_drivers_fullname_hash (cost=0.00..4.02
rows=3 width=0) (actual time=0.003..0.003 rows=1 loops=1)
        Index Cond: (((given_name)::text || ' '::text) ||
(family_name)::text) = 'Ayrton Senna'::text)
Planning Time: 0.022 ms
Execution Time: 0.011 ms
```

- **Análise:** O planejador passou a utilizar um Bitmap Index Scan no índice criado. A chave de busca foi mapeada instantaneamente no índice em memória, fornecendo a página física do registro. O PostgreSQL realizou um Bitmap Heap Scan para acessar apenas 1 único bloco físico na tabela Heap para retornar a nacionalidade, diminuindo drasticamente os tempos de CPU e I/O.

3. Exercício 2 — Cidades Brasileiras por Prefixo de Nome

A consulta busca retornar a latitude, longitude e população de cidades localizadas no Brasil cujo nome inicia com o prefixo "São" :

```
SELECT name::TEXT, latitude, longitude, population
FROM cities
WHERE country_id = (SELECT id FROM countries WHERE code = 'BR' LIMIT 1)
AND name LIKE 'São%';
```

3.1. Estado Inicial (Sem Índice)

- Tempo Médio por Execução: 6,10 ms
- Plano de Execução (EXPLAIN ANALYZE):

```
Seq Scan on cities (cost=8.16..1662.82 rows=1 width=56) (actual
time=0.173..6.097 rows=243 loops=1)
  Filter: (((name)::text ~ 'São%')::text) AND (country_id = $0)
  Rows Removed by Filter: 81026
  InitPlan 1 (returns $0)
    -> Limit (cost=0.14..8.16 rows=1 width=4) (actual time=0.007..0.007
rows=1 loops=1)
      -> Index Scan using countries_code_key on countries ...
Planning Time: 0.048 ms
Execution Time: 6.109 ms
```

- **Análise:** Embora o ID do Brasil (country_id) tenha sido resolvido rapidamente via busca indexada na tabela countries, o otimizador foi forçado a executar um Seq Scan varrendo as 81.269 linhas da tabela cities para aplicar os filtros. Isso exigiu processamento exaustivo de CPU e descartou 81.026 registros.

3.2. Escolha e Justificativa do Índice

Optamos pela criação de um **Índice B-Tree Composto com Cobertura (INCLUDE)**:

```
CREATE INDEX idx_cities_br_name_btree ON cities (country_id, name
text_pattern_ops)
INCLUDE (latitude, longitude, population);
```

- **Justificativa:** A busca por correspondência de prefixo (LIKE 'São%') exige uma estrutura ordenada, o que torna o uso do índice B-Tree obrigatório (índices Hash não suportam consultas por faixas ou padrões de texto).
- **Especificações do Índice:**
 - **Composto (country_id, name):** Mapeia de forma conjunta as duas condições do predicado.
 - **Classe de Operador text_pattern_ops:** Garante que o PostgreSQL ordene strings byte por byte (binário), viabilizando o uso correto da B-Tree na cláusula LIKE independentemente do locale ou codificação configurados no sistema operacional.
 - **Cláusula INCLUDE:** Adiciona as colunas projetadas (latitude, longitude, population) nas páginas folha do índice. Com isso, transformamos a estrutura em um índice de cobertura, permitindo que a consulta seja respondida inteiramente através de uma varredura de índice, sem a necessidade de acessar os dados reais na tabela física (Heap).

3.3. Estado Otimizado (Com Índice)

- **Tempo Médio por Execução: 0,08 ms** (Aceleração de aproximadamente **76 vezes**)
- **Plano de Execução (EXPLAIN ANALYZE):**

```
Bitmap Heap Scan on cities (cost=12.61..20.42 rows=2 width=56) (actual
time=0.031..0.077 rows=243 loops=1)
  Recheck Cond: (country_id = $0)
  Filter: ((name)::text ~~ 'São% '::text)
  Heap Blocks: exact=114
  InitPlan 1 (returns $0)
    -> Limit ...
    -> Bitmap Index Scan on idx_cities_br_name_btree (cost=0.00..4.44
rows=2 width=0) (actual time=0.023..0.023 rows=243 loops=1)
      Index Cond: ((country_id = $0) AND ((name)::text ~>=~
'São '::text) AND ((name)::text ~~<~ 'São '::text))
  Planning Time: 0.037 ms
  Execution Time: 0.089 ms
```

- **Análise:** O SGBD substituiu o Seq Scan de 81 mil linhas por um Bitmap Index Scan na B-Tree, localizando de forma logarítmica $O(\log N)$ o intervalo de chaves correspondente ao prefixo São. Em vez de varrer a tabela, leu apenas 114 blocos físicos contendo as cidades de interesse, reduzindo o tempo para menos de um décimo de milissegundo.

4. Exercício 3 — Questão Teórica (B-Trees e LIKE %valor%)

Pergunta

Estruturas B-trees conseguem indexar consultas com predicados do tipo: `<Atributo> LIKE "%valor%"` ? (Considere que o atributo seja do tipo TEXT). Explique.

Resposta

Não de forma eficiente.

A estrutura B-Tree mantém seus nós e chaves organizados em uma sequência estritamente ordenada (lexicograficamente no caso de textos). Isso viabiliza pesquisas rápidas em tempo logarítmico $O(\log N)$ por meio de busca binária baseada no início da chave (ex: LIKE 'São%').

Ao utilizar um curinga no início do padrão buscado (LIKE '%valor%'), o caractere inicial da string torna-se completamente desconhecido. Como a substring buscada pode iniciar com qualquer caractere, o SGBD perde o ponto de entrada na árvore e é incapaz de realizar a descida estruturada pelos nós. Nessas condições, o otimizador do PostgreSQL descarta a busca binária e realiza uma varredura sequencial de todas as chaves do índice (Index Full Scan) ou, mais comumente, varre a tabela inteira na Heap via Seq Scan, aplicando o filtro de correspondência de padrão linha por linha.

Solução Alternativa no PostgreSQL:

Para otimizar buscas por substrings genéricas, o PostgreSQL fornece suporte a índices invertidos GIN (Generalized Inverted Index) em conjunto com a extensão nativa pg_trgm (trigramas):

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;  
CREATE INDEX idx_cities_name_trgm ON cities USING GIN (name gin_trgm_ops);
```

O índice GIN fragmenta os textos em conjuntos de 3 caracteres (trigramas) e mapeia suas posições, permitindo buscas de alta performance com %valor% em frações de milissegundo.

5. Tabela Comparativa de Performance

A tabela abaixo consolida os resultados práticos coletados:

Cenário de Teste	Operação Principal	Estrutura Escolhida	Tempo Sem Índice	Tempo Com Índice	Fator de Speedup
Ex. 1 — Ayrton Senna	Igualdade Exata (=)	Hash Funcional	0,07 ms	0,01 ms	7 vezes
Ex. 2 — Cidades (Prefix)	Busca por Prefixo (LIKE)	B-Tree Composta	6,10 ms	0,08 ms	76 vezes
Ex. 3 — LIKE '%valor%'	Substring no Início	Inviável na B-Tree	—	—	—

6. Conclusão Geral

Os experimentos práticos evidenciaram que o uso de índices em SGBDs exige o alinhamento correto entre a estrutura física e o operador lógico do predicado. O índice Hash provou ser a melhor escolha para buscas pontuais de igualdade exata (Ex. 1), enquanto a B-Tree mostrou-se indispensável para consultas baseadas em intervalos e padrões ordenados (Ex. 2), especialmente quando potencializada por recursos como a classe de operador `text_pattern_ops` e a indexação de cobertura via cláusula `INCLUDE`.